MIT 6.s184 / 6.s978 Deep Generative Models
Problem Set 1 Report

Pierson dong

April 5, 2026

Problem 1: Auto-Encoder on MNIST

(a) AE Implementation

The encoder and decoder for the Auto-Encoder are implemented as follows:

```
# --- 1. Encoder ---
encoder_layers = []
in_dim = input_dim
for out_dim in hidden_dims:
    encoder_layers.append(torch.nn.Linear(in_dim, out_dim))
    if out_dim != hidden_dims[-1]:
        encoder_layers.append(torch.nn.ReLU())
    in_dim = out_dim
self.encoder = torch.nn.Sequential(*encoder_layers)

# --- 2. Decoder ---
decoder_layers = []
in_dim = hidden_dims[-1]
for out_dim in reversed(hidden_dims[:-1]):
    decoder_layers.append(torch.nn.Linear(in_dim, out_dim))
    decoder_layers.append(torch.nn.ReLU())
    in_dim = out_dim
decoder_layers.append(torch.nn.Linear(in_dim, input_dim))
decoder_layers.append(torch.nn.Sigmoid())
self.decoder = torch.nn.Sequential(*decoder_layers)
```

(b) AE Training and Evaluation

Problem 2: Variational Auto-Encoder on MNIST: SGVB

(a) Proof of the ELBO

We start from the marginal log-likelihood $\log p(x)$. By introducing a variational distribution $q(z|x)$ and applying Jensen's Inequality (since the logarithmic function is concave), we derive the ELBO:

$$\begin{aligned}
\log p(x) &= \log \int p(x,z)\,dz \\
&= \log \int q(z|x)\frac{p(x,z)}{q(z|x)}\,dz \\
&= \log \mathbb{E}_{q(z|x)}\left[\frac{p(x,z)}{q(z|x)}\right] \\
&\geq \mathbb{E}_{q(z|x)}\left[\log \frac{p(x,z)}{q(z|x)}\right] \quad \text{(by Jensen's Inequality)}
\end{aligned}$$

Thus, the Evidence Lower Bound (ELBO) is $\mathbb{E}_{q(z|x)}\left[\log \frac{p(x,z)}{q(z|x)}\right]$.

Figure 1: AE Reconstruction and Interpolation Results on MNIST.

(b) VAE Encoder and Decoder

```
# --- 1. Encoder ---
encoder_layers = []
in_dim = input_dim
for i, out_dim in enumerate(hidden_dims):
    encoder_layers.append(torch.nn.Linear(in_dim, out_dim))
    if i < len(hidden_dims) - 1:
        encoder_layers.append(torch.nn.ReLU())
    in_dim = out_dim
self.encoder = torch.nn.Sequential(*encoder_layers)

# --- 2. Decoder ---
decoder_layers = []
in_dim = self.z_size
for out_dim in reversed(hidden_dims[:-1]):
    decoder_layers.append(torch.nn.Linear(in_dim, out_dim))
    decoder_layers.append(torch.nn.ReLU())
    in_dim = out_dim

final_out_dim = input_dim if decode_dim == -1 else decode_dim
decoder_layers.append(torch.nn.Linear(in_dim, final_out_dim))

if use_sigmoid:
    decoder_layers.append(torch.nn.Sigmoid())
```

```
self.decoder = torch.nn.Sequential(*decoder_layers)
```

(c) Reparameterization Trick

```
def reparameterize(self, mean, logvar, n_samples_per_z=1):
    std = torch.exp(0.5 * logvar)
    eps = torch.randn_like(std)
    z = eps * std + mean
    return z
```

(d) SGVB log_normal_pdf

```
def log_normal_pdf(sample, mean, logvar, raxis=1):
    log_p = -0.5 * (log2pi + logvar + ((sample - mean) ** 2) / torch.exp(logvar
    ))
    return torch.sum(log_p, dim=raxis)
```

Problem 3: VAE on MNIST: Analytical KL Divergence

(a) Proof of ELBO Decomposition

Given $p(z) = \mathcal{N}(z; 0, I)$ and $q(z|x) = \mathcal{N}(z; \mu(x), \mathrm{diag}(\sigma^2(x)))$:

$$\begin{aligned}
\text{ELBO} &= \mathbb{E}_{q(z|x)}\left[\log \frac{p(x,z)}{q(z|x)}\right] \\
&= \mathbb{E}_{q(z|x)}[\log p(x|z) + \log p(z) - \log q(z|x)] \\
&= \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{KL}(q(z|x) \parallel p(z))
\end{aligned}$$

The KL divergence between a multivariate Gaussian with diagonal covariance and a standard normal distribution decomposes dimension-wise. For each dimension d:

$$\begin{aligned}
-D_{KL}(q(z_d|x) \parallel p(z_d)) &= \int q(z_d|x) \log \frac{p(z_d)}{q(z_d|x)} dz_d \\
&= \mathbb{E}_q\left[-\frac{1}{2}\log(2\pi) - \frac{1}{2}z_d^2 - \left(-\frac{1}{2}\log(2\pi\sigma_d^2) - \frac{(z_d-\mu_d)^2}{2\sigma_d^2}\right)\right] \\
&= \mathbb{E}_q\left[\frac{1}{2}\log(\sigma_d^2) - \frac{1}{2}z_d^2 + \frac{(z_d-\mu_d)^2}{2\sigma_d^2}\right]
\end{aligned}$$

Since $\mathbb{E}_q[z_d^2] = \mu_d^2 + \sigma_d^2$ and $\mathbb{E}_q[(z_d - \mu_d)^2] = \sigma_d^2$:

$$\begin{aligned}
-D_{KL}(q(z_d|x) \parallel p(z_d)) &= \frac{1}{2}\log(\sigma_d^2) - \frac{1}{2}(\mu_d^2 + \sigma_d^2) + \frac{1}{2} \\
&= \frac{1}{2}\left(1 + \log(\sigma_d^2) - \mu_d^2 - \sigma_d^2\right)
\end{aligned}$$

Summing over all dimensions d gives:

$$-D_{KL}(q(z|x) \parallel p(z)) = \frac{1}{2}\sum_d \left(1 + \log(\sigma_d^2) - \mu_d^2 - \sigma_d^2\right)$$

This yields the required analytical decomposition.

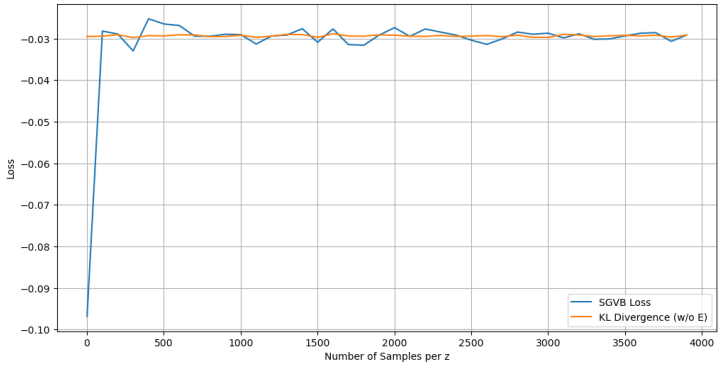

Figure 2: SGVB Estimator vs Analytical KL Divergence.

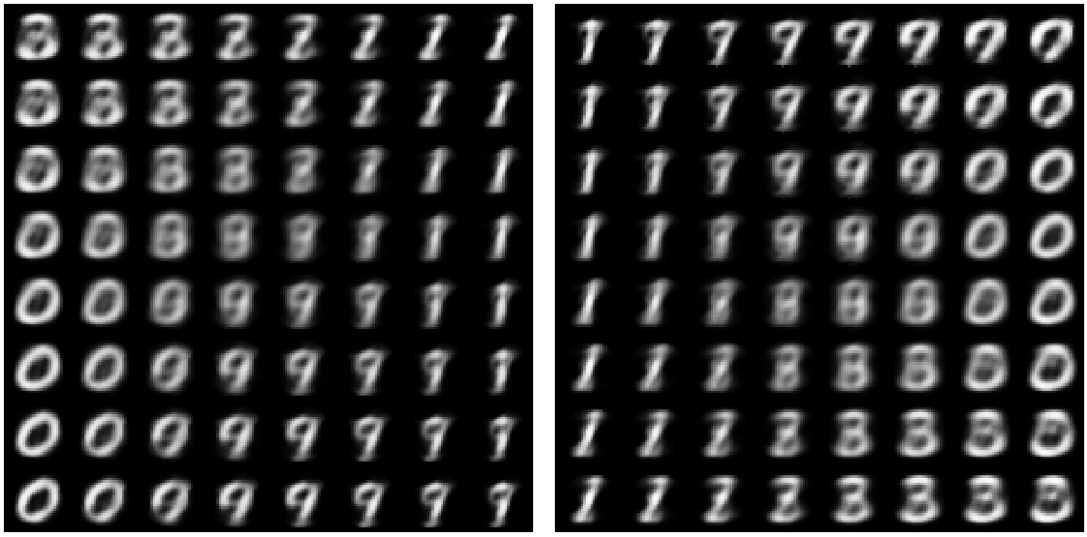

Figure 3: VAE Results using SGVB (Left) and Analytical KL (Right).

(b) SGVB vs Analytical KL Curves

(c) Training VAEs with both losses

Problem 4: VAE on Torus Point Cloud

(a) Reconstruction Results

(b) Linear Interpolation

(c) Architecture Modification for Intrinsics (Optional)

To better capture the intrinsic toroidal structure of the data, the architecture utilizes a **Positional Encoding** layer before the VAE encoder. This transforms the original 3D coordinates (x, y, z) into a high-dimensional feature space using sine and cosine functions across multiple frequency bands.

By directly initializing the VAE with these multi-frequency inputs inside the `PointVAE` module (with `use_sigmoid=False`), the neural network overcomes spectral bias. This effectively enforces the decoded interpolation trajectory to adhere to the curved geometry of the Torus, correctly mapping linear traversals in the latent space to geodesic paths along the surface of the 3D object.

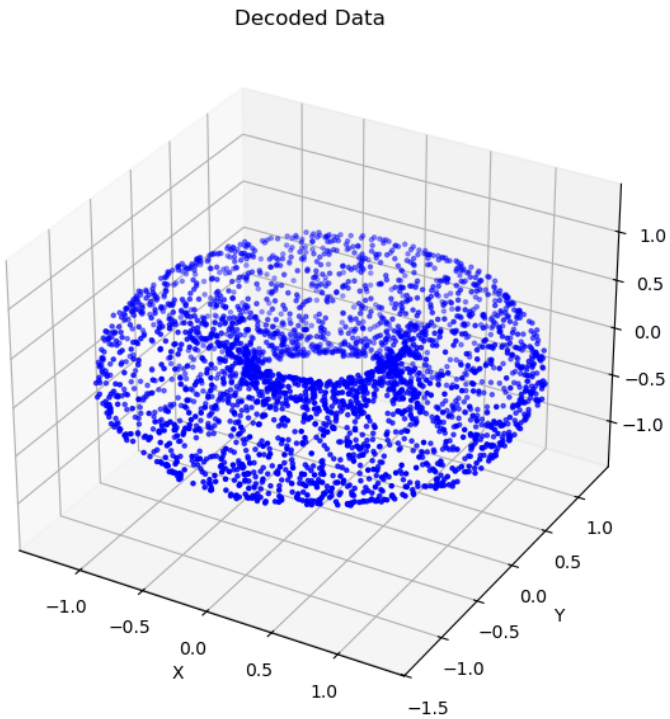

Figure 4: Torus Point Cloud Reconstruction.

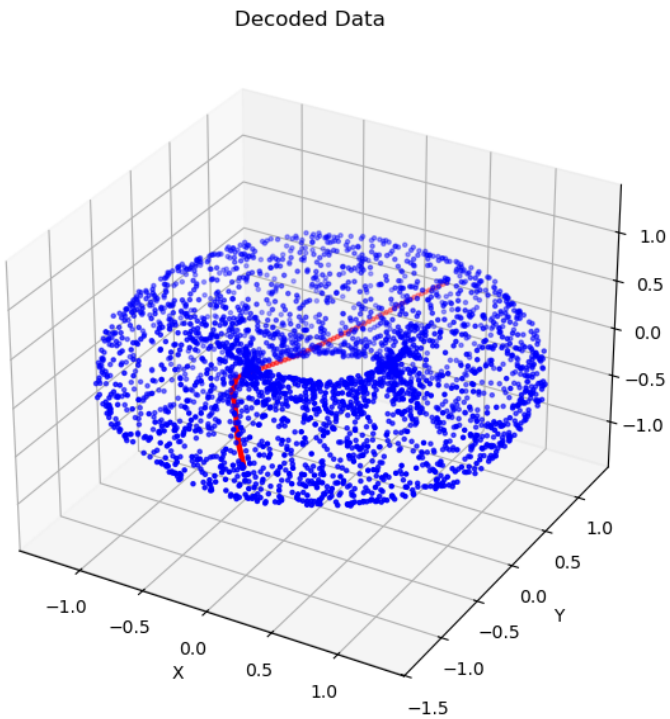

Figure 5: Linear Interpolation Output mapped to Torus.